

Decentralised Lending Service — Dettaglio implementativo

Peer2Peer Systems and Blockchains — A.A. 2025/26
Walkthrough completo file-per-file del codice consegnato.

Questo documento riprende e dettaglia tutti i file del progetto, in modo che un lettore che non l'abbia mai visto possa capire ogni componente solo leggendolo. Per ogni file viene spiegato: scopo, ruolo nell'architettura, stato/variabili, ogni funzione rilevante, e i punti dove la logica realizza una specifica del documento di consegna (*docs/v011_ProjectP2PBC_LAB2526.pdf*, citato come **Spec §x.y**).

Sommario

- 1. Inquadramento e mappa del progetto
- 2. Costanti di sistema e relazione con la Spec
- 3. Genesis, rete privata e setup Hardhat
- 4. Smart contract: BitcoinOracle.sol
- 5. Smart contract: LendingPool.sol
- 6. Smart contract: LoanContract.sol
- 7. LocalProxy.sol — proxy UUPS locale
- 8. Mocks (MockBitcoinOracle, LendingPoolV2)
- 9. Contratti vulnerabili (Reentrancy demo)
- 10. Oracolo off-chain — oracle/oracle_service.py
- 11. Script Python: InitialSetup.py
- 12. Script Python: DemoOperations.py
- 13. Script Python: AutoVoter.py
- 14. Script Python: GasMeasurement.py
- 15. Suite di test Hardhat
- 16. Flusso end-to-end di un prestito
- 17. Misure di gas
- 18. Reentrancy: vulnerabilità e difesa
- 19. Strategie malevole (non-reentrancy)
- 20. Manuale d'uso operativo

1. Inquadramento e mappa del progetto

Il progetto implementa un **servizio di prestito decentralizzato** su Ethereum (Spec §1). Tre attori interagiscono on-chain:

- **Contributori:** depositano ETH in una *funding pool* comune, votano le proposte di prestito, riscuotono interessi e possono chiedere compensazione per prestiti falliti.
- **Applicanti:** chiedono prestiti specificando importo, tasso (1–100%), durata in blocchi e un indirizzo Bitcoin come prova di liquidità.
- **Operatore dell'oracolo:** account autorizzato a scrivere on-chain il saldo dell'indirizzo Bitcoin dopo aver letto blk*.dat.

Struttura cartelle

contracts/	
BitcoinOracle.sol	– endpoint on-chain dell'oracolo BTC
LendingPool.sol	– pool principale (UUPS upgradeable)
LoanContract.sol	– contratto per-prestito (deployato a runtime)
LocalProxy.sol	– ERC1967Proxy ricompilato localmente
mocks/	
MockBitcoinOracle.sol	– oracolo finto usato nei test JS
LendingPoolV2.sol	– implementazione v2 per test di upgrade
vulnerable/	
LendingPoolVulnerable.sol	– versione bacata di withdraw() (demo §1.5)
ReentrancyAttacker.sol	– contratto attaccante
oracle/	
oracle_service.py	– daemon off-chain che ascolta UpdateRequested
scripts/	
InitialSetup.py	– crea account, deploia tutto
DemoOperations.py	– demo end-to-end 16 step
AutoVoter.py	– bot 'approve everything'
GasMeasurement.py	– misura gas per ogni operazione
test/	– Hardhat test suite (.test.js)
project2526genesis.json	– genesis Clique PoA, chainId 202526
hardhat.config.js	– solc 0.8.22, runs=200
data/	– keystore + JSON output di runtime

Sintesi del rapporto fra contratti

LendingPool è il fulcro: tiene la contabilità globale (deposits, lockedValue, compensationPool, collateralPercentage). **BitcoinOracle** è uno store di saldi Bitcoin (in satoshi) interrogato da LendingPool al momento della resolution per il check di liquidità. Ogni proposta approvata istanzia un nuovo **LoanContract** che gestisce in modo isolato il singolo prestito (rimborso, fallimento, compensazione). LoanContract è registrato in LendingPool tramite la mappa *isActiveLoan*; solo i contratti registrati possono richiamare gli hooks (lockValue, repayLockedValue, creditInterest, ...).

2. Costanti di sistema (Spec §1.3)

Nome	Valore	Dove sta	Cosa significa
MIN_DEPOSIT	100 000 wei	LendingPool	deposito minimo (Spec §1.3)
PROPOSAL_VOTING_PERIOD	200 blocchi	LendingPool	minimo da attendere prima di resolveProposal
INITIAL_COLLATERAL_PERCENT	50	LendingPool	% iniziale collaterale; range 1–100
COLLATERAL_STEP	5	LendingPool	± ad ogni loan failed / successful
MIN_ORACLE_FEE	$45\,667 \times 0.1$ gwei	BitcoinOracle	fee minima per requestUpdate
BTC_ETH_RATE	30	BitcoinOracle	tasso fisso 1 BTC = 30 ETH
SATOSHI_PER_BTC	1e8	BitcoinOracle	denominatore conversione
MAX_BLOCKS (oracolo)	131 000	oracle_service.py	limite blocchi BTC (Spec §1.4)

Tutte le durate sono in **numero di blocchi**, non secondi (Spec §1.3 nota): un "tempo 10" significa block.number che cresce di 10.

3. Genesis, rete privata e Hardhat

La rete privata usa **Clique PoA** con periodo 10 s. Il file *project2526genesis.json* dichiara:

```
chainId      : 202526
consensus    : clique (period=10, epoch=30000)
gasLimit     : 60_000_000
sealer/etherbase : 0xd278d247A52C550508ea2b2C9321d816238fb523
alloc (sealer)  : 3·1036 wei (genesis prefunded)
```

Lo stesso indirizzo del sealer è anche l'unico account prefondato. **Vincolo Spec §1.5:** il sealer può *solo* trasferire ETH ad altri account, non deploya contratti né esegue altre transazioni. Lo script *InitialSetup.py* rispetta il vincolo: usa la chiave del sealer solo in *send_eth(...)* per finanziare gli account creati, e tutte le deploy partono da un account nuovo (deployer / oracle_operator).

hardhat.config.js:

```
solidity: version "0.8.22", optimizer { enabled: true, runs: 200 }
networks:
  hardhat: chainId 202526 (in-memory per i test)
  privatechain: url http://127.0.0.1:8545 chainId 202526
```

4. contracts/BitcoinOracle.sol

Componente **on-chain** dell'oracolo (Spec §1.4). Memorizza il saldo di ogni indirizzo Bitcoin in *satoshi*, indicizzato dal hash keccak256 della stringa dell'indirizzo.

Stato

```
uint256 constant MIN_ORACLE_FEE = 45667000000000; // 45667 * 0.1 gwei
uint256 constant BTC_ETH_RATE   = 30;
uint256 constant SATOSHI_PER_BTC = 1e8;
address immutable operator;      // = msg.sender al deploy
mapping(bytes32 => uint256) balances; // hash(btcAddr) -> satoshi
```

L'operatore (immutable) è il deployer ed è l'unico autorizzato a chiamare *update*. La fee 45667×0.1 gwei è dimensionata sul gas di una SSTORE su slot zero (Gsset 20000): vedi *test/OracleFee.test.js* che verifica empiricamente l'uguaglianza.

Funzioni

requestUpdate(bytes32 btcHash) payable — chiunque la chiama, paga la fee, e l'evento *UpdateRequested* viene emesso. Il daemon Python ne intercetterà l'occorrenza per pubblicare il saldo.

update(bytes32 btcHash, uint256 satoshi) onlyOperator — scrive il saldo on-chain ed emette *BalanceUpdated*.

getBalance(hash) view — ritorna i satoshi memorizzati.

getEthEquivalent(hash) view — conversione satoshi $\rightarrow$ wei: $balance \times 30 \times 1e18 / 1e8$. È la funzione interrogata da *LendingPool.resolveProposal* per il check di liquidità (Spec §1.4: se BTC equivalent $\geq$ amount, check passa).

hashBtcAddress(string) — utility pure: keccak256 della stringa.

withdrawFees() onlyOperator — drena le fee accumulate verso l'operatore.

Eventi

```
UpdateRequested(bytes32 indexed btcHash, address indexed requester);
BalanceUpdated(bytes32 indexed btcHash, uint256 newBalance);
```

5. contracts/LendingPool.sol

Contratto principale, **UUPS-upgradeable** via OpenZeppelin (*Initializable*, *UUPSUpgradeable*, *OwnableUpgradeable*). Tiene contabilità del pool, gestisce proposte e vota, deploya i LoanContract approvati, espone gli hooks per quei prestiti.

Stato

```
IBitcoinOracle public oracle;
uint256 public totalFundingPool; // somma deposits (anche lockati)
uint256 public totalLocked; // somma lockedValue
uint256 public compensationPool; // pool di compensazione (Spec §1.2)
uint256 public collateralPercentage; // 50 a init
mapping(address => uint256) public deposits;
mapping(address => uint256) public lockedValue;
mapping(address => bool) public isActiveLoan; // registro loan attivi
address[] private _contributorList; // ordinato per arrivo
mapping(address => bool) private _contributorTracked;

struct Proposal {
    address applicant; uint256 amount; uint8 interestRate;
    uint256 duration; bytes32 btcAddressHash; uint256 submittedBlock;
    ProposalStatus status; address[] approveVoters;
    mapping(address => bool) hasVoted;
    mapping(address => bool) voteApprove;
}
mapping(uint256 => Proposal) internal _proposals;
uint256 public proposalCount;
```

Nota. Le mapping nidificate dentro Proposal impediscono di esporla via getter pubblico, quindi sono esposti tre view dedicati: *getProposal*, *hasVotedOn*, *getVoteApprove*.

Init / costruttore

Il costruttore disabilita gli initializer (pattern UUPS): *_disableInitializers()*. Lo stato vivo è creato da *initialize(address oracle)*, chiamata **una sola volta** attraverso il proxy ERC1967 al momento del deploy. Setta owner, *_reentrancyStatus*=1, oracle, collateralPercentage=50.

Modificatori

nonReentrant — guard manuale (in OZ v5 il modulo non è più disponibile per i contratti upgradeable). Usa *_reentrancyStatus* 1/2. Applicato a *deposit*, *withdraw*, *resolveProposal*, *compensateFromPool*.

onlyActiveLoan — richiede che *msg.sender* sia registrato come loan attivo. Protegge tutti gli hooks invocabili dai LoanContract.

Views

disposableValue(addr) = deposits[addr] – lockedValue[addr] (Spec §1.3: "disposable value" è il valore non bloccato).

totalDisposable() = totalFundingPool – totalLocked.

isContributor(addr) = deposits[addr] > 0.

Operazioni contributori

deposit() payable nonReentrant — richiede *msg.value* ≥ MIN_DEPOSIT, aggiunge il mittente alla *_contributorList* se nuovo, aggiorna deposits e totalFundingPool, emette *Deposited*.

withdraw(uint256 amount) nonReentrant — verifica *disposableValue* ≥ *amount*, sottrae **prima** da deposits e totalFundingPool, poi esegue il *call{value:amount}* (checks-effects-interactions corretto). Emette *Withdrawn*.

Interazione con l'oracolo

requestOracleUpdate(bytes32 btcHash) payable — forward della fee alla funzione omonima di BitcoinOracle. Il pool stesso non memorizza saldi, ma fa da "facade" per facilitare l'utente.

Proposal lifecycle

submitProposal(amount, interestRate, duration, btcHash) — valida i campi ($\text{amount} > 0$, $1 \leq \text{rate} \leq 100$, $\text{duration} > 0$), inserisce un nuovo Proposal in `_proposals`, incrementa `proposalCount`, salva il block di submission, `status=Active`. Emette `ProposalSubmitted`. Notabile: *chiunque* può sottoporre (anche non-contributor).

vote(proposalId, bool approve) — solo contributori; chi non vota è considerato implicit reject (Spec §1.3). Una sola votazione per contributore. Se `approve=true`, il voter è aggiunto a `approveVoters` (serve per la weighted sum a resolve time).

resolveProposal — cuore del sistema

Chiamabile solo dall'*applicant* originale, e solo dopo che `block.number > submittedBlock + PROPOSAL_VOTING_PERIOD` (strict). L'esecuzione è una catena di early-rejection seguita, in caso favorevole, dal lock proporzionale e dal deploy del LoanContract.

Step A — early-reject pool low: se `totalDisposable < amount`, `status=Rejected`, `ProposalRejected`, return.

Step B — early-reject BTC: se `oracle.getEthEquivalent(btcHash) < amount`, `status=Rejected`, return (Spec §1.4).

Step C — weighted vote: somma `disposableValue` di ciascun `approveVoter`. Se `weightedYes × 2 ≤ totalDisp` (tie incluso → reject, Spec §1.3) `status=Rejected`, return.

Step D — lock proporzionale: per ogni contributor `c` con `disp > 0`, `share_c = floor(amount × disp_c / totalDisp)`. Skippa i contributor con `share=0` per risparmiare gas. `loanedAmount = Σ share_c` può essere $< \text{amount}$ (eventuale dust da floor è dedotta dal prestito, non creditata all'applicant — Spec §1.3).

Step E — sort: `_sortContributors` ordina le coppie (`addr, share`) **DESC per share, tie-break ASC per address**. L'ordine è cruciale: LoanContract userà esattamente questo ordine come "waterfall" per rimborsare il base amount.

Step F — apply locks + deploy loan: aggiunge `share_c` a `lockedValue[c]`, `totalLocked`. Poi:

```
address loanAddr = address(
  new LoanContract{value: loanedAmount}(
    p.applicant, loanedAmount, collateralPercentage,
    block.number + p.duration, finalAddr, finalShares
  )
);
isActiveLoan[loanAddr] = true;
emit LoanRegistered(loanAddr);
emit ProposalApproved(proposalId, loanAddr, loanedAmount);
```

Hooks per LoanContract (onlyActiveLoan)

lockValue / unlockValue — aggiorna `lockedValue + totalLocked`.

repayLockedValue(c, amount) payable — il loan trasferisce ETH indietro al pool a fronte di un rimborso base. Decrementa `lockedValue` e `totalLocked`. *deposits[c] non cambia*: i depositi non erano stati decrementati al lock.

creditInterest(c) payable — il loan inoltra ETH del gain *direttamente* al contributor (non nel pool — Spec §1.3).

addToCompensationPool() payable — accumula collateral e residui.

compensateFromPool(c, amount) — drena dal `compensationPool` verso `c`. Side-effect importante: *riduce deposits[c] e lockedValue[c] di amount* (la quota persa nel prestito fallito viene "chiusa" dal pool). Reverte se `amount > compensationPool / deposits / lockedValue`.

increaseCollateral / decreaseCollateral — ± 5 con clamp `[1, 100]`. Chiamate da LoanContract sulla transizione `fail/success`.

markLoanClosed — il loan si auto-deregistra (Spec §1.5 "properly manage termination").

UUPS

`_authorizeUpgrade(address) internal override onlyOwner` — garantisce che solo l'owner possa firmare un upgrade. Vedi *test/Upgradability.test.js* per la copertura.

6. contracts/LoanContract.sol

Contratto deployato da *LendingPool.resolveProposal* per ogni prestito approvato (Spec §1.5: "un nuovo loan attivo deve essere gestito da uno smart contract dedicato appena deployato"). Gestisce in modo isolato disbursement, rimborsi, compensazione e termination.

Stato immutable

```
address immutable applicant;  
uint256 immutable loanedAmount;  
uint256 immutable collateralPercentage; // snapshot al deploy  
uint256 immutable expiryBlock;  
ILendingPool immutable lendingPool; // = msg.sender al deploy
```

Tutti immutable: una volta deployato il loan, i suoi parametri non cambiano più. In particolare *collateralPercentage* è fissata al valore attuale del pool, non al valore al close — questo è esplicitato dalla Spec §1.3.

Stato mutabile

```
struct Contributor { address addr; uint256 initialLocked; }  
Contributor[] public contributors; // ordine waterfall (DESC share)  
uint256 public totalInitialLocked;  
mapping(address => uint256) public initialLockedOf; // 0(1) lookup  
mapping(address => uint256) public unlockedSoFar;  
mapping(address => uint256) public alreadyCompensated; // cumulato pagato dal comp-pool  
mapping(address => uint256) public compRecovered; // recuperato a favore del comp-pool  
uint256 public remainingLoanAmount;  
Status public status; // Active | Failed | Successful  
bool public terminated; // selfdestruct-flag (vedi §1.5)
```

Costruttore

Valida tutti i parametri (length match, no zero, sum delle locks == loaned), popola *contributors* e *initialLockedOf*, salva applicant ecc. e **disburse l'intero loanedAmount all'applicant** via call. *msg.value* deve essere esattamente loanedAmount (LendingPool lo passa nel *new LoanContract{value:...}*).

Importante: *lendingPool = msg.sender*, quindi solo LendingPool potrà richiamare *markFailed*.

partialRepay()

Solo l'applicant, mentre *status* ∈ {Active, Failed}, non terminato. Quattro step:

Step 1: *split baseAmount = min(msg.value, remaining)*, *interest = msg.value – baseAmount*. Tutto ciò che eccede il base è interest (Spec §1.3: "may repay even more than original amount").

Step 2 — Waterfall base: itera contributors nell'ordine fissato (DESC *initialLocked*, tie ASC address). Per ognuno calcola *capacity = initialLocked – unlockedSoFar – compRecovered*; prende *take = min(baseRemaining, capacity)*; lo splitta in due via *_splitBaseForfeit*: se il contributore aveva già ricevuto compensazione (*alreadyCompensated > compRecovered*), una frazione proporzionale all'*outstanding* rispetto alla *remainingShare* torna al comp pool, il resto va in *repayLockedValue* sul pool. Quando un contributor è saturato (*take == capacity*) il floor è esatto e l'avanzo del pool viene completamente recuperato in quella chiamata.

Step 3 — Interesse: *collateralAmount = interest × collateralPct / 100* va tutto al compPool. *gain = interest – collateral* è distribuito direttamente ai contributor in proporzione a *initialLocked / totalInitialLocked* via *creditInterest*. Per ogni contributor con *alreadyCompensated > 0*, una frazione costante del gain (= *ac / initialLocked*) è dirottata al comp pool come bonus proporzionale al rischio assorbito (Spec §1.3).

Step 4 — Chiusura: decrementa *remainingLoanAmount*. Se va a 0:

- Esegue un "residue pass" difensivo che chiude eventuali gap di *alreadyCompensated – compRecovered* e qualsiasi dust di *initialLocked – unlockedSoFar – compRecovered*.
- Forwarda *toComp* e fa uno sweep del balance residuo al comp pool.
- Se il loan non era Failed, *transition → Successful*, chiama *decreaseCollateral* e *markLoanClosed* sul pool, emette *LoanClosed*. **Un loan Failed resta Failed** anche se viene rimborsato per intero (Spec §1.3).

requestCompensation()

Chiamabile da qualunque contributor del loan se `status != Successful` e o `status==Failed` o (`status==Active` and `expired` and `remaining>0`). Sulla prima chiamata `transition` → `Failed`, chiama *increaseCollateral* (+5) — questa è la *marcatatura come fallito* di Spec §1.3.

Calcola $owed = initialLocked - unlockedSoFar - alreadyCompensated$ e $paid = \min(owed, compPool.balance)$. Aggiorna $alreadyCompensated += paid$ **prima** della chiamata esterna (*compensateFromPool*) per impedire reentrancy. Se `compPool` è basso, paga meno di `owed` e l'invito a richiamare in futuro è esplicito (Spec §1.3: "può essere chiamata più volte").

markFailed()

Hook `onlyLendingPool`: oggi non viene mai usato (la transizione a `Failed` avviene tutta in *requestCompensation*), ma è esposto come *surface* stabile per estensioni future.

terminate()

Cleanup `permissionless` (chiunque può chiamarla) richiesto da Spec §1.5 ("properly manage contracts termination"). Pre-condizioni:

- `status==Successful` → ok;
- `status==Failed` con tutti gli `owed = 0` (ogni quota o rimborsata o compensata) → ok;
- `status==Active` → revert *Loan still active*.

Forwarda eventuali residui al `comp pool`, deregistra il loan dal pool, alza il flag *terminated*. Il modifier *notTerminated* blocca ogni chiamata mutante successiva. Non si usa *selfdestruct* perché EIP-6049 post-Cancun non cancella più storage; il flag fornisce semantica equivalente.

Helper interni

_splitBaseForfeit(c, share): split del waterfall base (*toComp*, *toC*). Cap di sicurezza a outstanding e share.

_splitGainForfeit(c, g): split del gain (*gComp*, *gC*) con rapporto costante *ac/il*.

Nota. Il contratto **non ha `receive()` né `fallback()`**: ogni `wei` in ingresso deve passare da `partialRepay` (`payable`). I trasferimenti diretti rivertano. L'unica iniezione residua possibile è da `selfdestruct` di un altro contratto, gestita dallo sweep finale di *terminate()*.

7. contracts/LocalProxy.sol

Wrapper banale che re-importa *ERC1967Proxy* di OpenZeppelin per **ricompilarlo localmente** con la stessa evmVersion del progetto (*paris* per solc 0.8.22, default). L'artifact precompilato in *node_modules* usa evmVersion *shanghai* ed emette PUSH0; su chain pre-Shanghai (la genesis usa Berlin) si avrebbe il runtime *invalid opcode: PUSH0*. Il *LocalERC1967Proxy* elimina il problema.

```
contract LocalERC1967Proxy is ERC1967Proxy {
    constructor(address impl, bytes memory data) payable
        ERC1967Proxy(impl, data) {}
}
```

8. Mocks

contracts/mocks/MockBitcoinOracle.sol

Versione minimale dell'oracolo per i test JS: niente fee, niente operator. Solo *setEthEquivalent* / *getEthEquivalent* / *requestUpdate*. Permette ai test di simulare facilmente "BTC liquidity check passes/fails".

contracts/mocks/LendingPoolV2.sol

Implementazione v2 finta usata da *Upgradability.test.js*. Eredita *LendingPool*, aggiunge *uint256 extraSlot* e *version()*. Espone *initializeV2()* *reinitializer(2)* (no-op di esempio) richiesta dal plugin OZ upgrades.

9. Contratti vulnerabili (demo Reentrancy)

contracts/vulnerable/LendingPoolVulnerable.sol

Clone di *LendingPool* con due modifiche solo in *withdraw()*:

- 1) **rimosso** il modifier *nonReentrant*;
- 2) **CEI invertito**: la *call* avviene **prima** di aggiornare *deposits* e *totalFundingPool*;
- 3) gli aggiornamenti sono dentro *unchecked* per riprodurre il comportamento pre-0.8 (silent underflow), altrimenti la default checked arithmetic di 0.8 farebbe revert nello unwind dello stack e maschererebbe il bug.

Tutte le altre funzioni sono identiche al contratto sicuro.

contracts/vulnerable/ReentrancyAttacker.sol

Pattern classico DAO-style: contributor con piccolo deposito, *withdraw()* + ri-entrata da *receive()* finché *attackCount* < *MAX_REENTRIES* (=5) e il pool ha ancora fondi.

```
function deposit() external payable {
    attackAmount = msg.value;
    pool.deposit{value: msg.value}();
}
receive() external payable {
    if (attackCount < MAX_REENTRIES && address(pool).balance >= attackAmount) {
        attackCount++;
        pool.withdraw(attackAmount);
    }
}
function attack() external { pool.withdraw(attackAmount); }
```

10. oracle/oracle_service.py

Demone off-chain dell'oracolo. Tre fasi:

Fase 1 — parse blk.dat: `load_all_blocks()` legge i file `chaindata/blk00000.dat` e `blk00001.dat` (cap `MAX_BLK_FILES=2`). Riconosce il magic number Bitcoin mainnet `0xf9beb4d9`, deserializza ogni blocco con `bitcoin.core.CBlock` e salva (`prevhash`, `block`) in un dizionario indicizzato dall'hash del blocco.

Fase 2 — ordina per altezza: `sort_blocks_by_height()` parte dal genesis (`prevhash` tutto-zero) e segue la catena `prev→hash` fino a `MAX_BLOCKS=131000` (limite imposto da Spec §1.4).

Fase 3 — costruisce UTXO set: `process_block()` elabora **un blocco alla volta** (Spec §1.4: "as if new Bitcoin blocks were generated as normal"). Per ogni input rimuove l'UTXO speso e decrementa il saldo dell'address; per ogni output, estrae l'address con `extract_address(scriptPubKey)` e aggiunge un nuovo UTXO. Finalizza in `hashed_balances`: `keccak256(addr_string) → satoshi`, esattamente la chiave usata on-chain.

Fase 4 — listener: `listen_to_requests()` crea un filter sugli eventi `UpdateRequested` emessi da `BitcoinOracle`, e per ogni richiesta costruisce/firma/invia una `oracle.update(hash, satoshi)` on-chain con la chiave `OPERATOR_PRIVATE_KEY` (env). Polling ogni 2 s.

Nota. Path resolution rispetto a `__file__` rende lo script eseguibile da qualsiasi cwd. Il file di config `data/oracle_contract_info.json` è prodotto da `InitialSetup.py`.

11. scripts/InitialSetup.py

Bootstrap one-shot della chain privata, richiesto dalla Spec §1.5. Sequenza:

Step 0 — pulisce eventuali JSON residui di run precedenti (`accounts.json`, `oracle_contract_info.json`, `lending_pool_info.json`).

Step 1 — crea `N+M+3` account EOA con `eth_account.Account.create()`: `deployer`, `oracle_operator`, `auto_voter`, `N` contributori (default 3), `M` applicanti (default 2). Tutti tunable via env.

Step 2 — sblocca il keystore del sealer (decrypt JSON con password `project2526` letta da file) e **usa la sua chiave solo per send_eth** (Spec §1.5: "prefunded account can ONLY transfer value"). Funding di default: `deployer` 10 ETH, `operator` 1 ETH, `auto_voter` 5 ETH, ciascun contributore 5 ETH, ciascun applicant 1 ETH.

Step 3 — deploy **BitcoinOracle** dal `oracle_operator` (così l'operator immutable corrisponde). Sanity check: legge `oracle.operator()` on-chain e asserisce.

Step 4 — deploy **LendingPool implementation** dal `deployer`; encoda i calldata di `initialize(oracleAddr)`; deploia il **LocalERC1967Proxy(impl, initData)**. Verifica che attraverso il proxy si legga `collateralPercentage=50`, `owner=deployer`, `oracle` corretto.

Step 5 — scrive in `data/`:

```
accounts.json           - address+key di TUTTI gli account creati
oracle_contract_info.json - address + abi di BitcoinOracle
lending_pool_info.json  - proxy + impl + abi di LendingPool
```

Questi file sono input per `oracle_service.py`, `DemoOperations.py`, `AutoVoter.py`, `GasMeasurement.py`.

Nota. Sicurezza: `accounts.json` contiene chiavi private — è solo per *private chain didattica*, non andrebbe mai commitato in repo reali.

12. scripts/DemoOperations.py

Demo end-to-end in 16 step. Risponde direttamente al requisito Spec §1.5: "Provide a python script for executing an example set of operations [...] including printing the balance of the involved accounts after any meaningful change". Tutto l'output è duplicato anche su *data/demo_log.txt* via la classe *Tee*.

Step (sintesi)

#	Cosa fa
1	Stato iniziale: stampa pool state + balance di contrib/applicant.
2	3 deposit da 1, 2, 3 ETH. Verifica deposits / totalFundingPool.
3	Partial withdraw 0.3 ETH dal contrib[0].
4	requestOracleUpdate (pagando MIN_ORACLE_FEE) per l'indirizzo BTC genesis 1A1z...vfNa; attende BalanceUpdated dall'oracolo
5	submitProposal: 1 ETH, rate 20%, durata 40 blocchi, btcHash.
6	Voting: contrib[0] REJECT, contrib[1]/[2] APPROVE (pesi 0.7/2/3 → yes weight 5 > no weight 0.7 → passa).
7	mine_blocks(PROPOSAL_VOTING_PERIOD+1) — su geth Clique attende passivamente; su hardhat usa hardhat_mine.
8	resolveProposal → ProposalApproved → LoanContract istanziato.
9	Ispezione LoanContract: applicant, loanedAmount, expiry, contributors con initialLocked.
10	partialRepay(0.4 ETH): aggiorna remaining; stampa stato loan.
11	partialRepay finale: remaining + interest 20% → loan Successful, collateralPercentage 50→45.
12	Loan 2 (applicant[1], 0.6 ETH, 30%, durata 15 blocchi): approvato; mining di 16 blocchi per andare oltre l'expiry senza che l'applicant
13	requestCompensation dal contrib[2] (quota maggiore): transition Failed, collateral +5, paga min(owed, compPool).
14	Late partialRepay 0.25 ETH sul loan Failed: waterfall riempie prima il contrib di quota maggiore.
15	Seconda requestCompensation dopo il refill — verifica che MarkedFailed NON si riemette e collateralPct non cambia (spec: marcatur
16	Stato finale: pool, contributori, applicanti, isActiveLoan.

Funzioni utility importanti: *send_tx* (firma+invia+attende receipt; exit su status≠1), *mine_blocks* (tenta hardhat_mine/evm_mine, poi fallback a polling), *print_pool_state* / *print_contributor_state* / *print_loan_state*, *wait_for_balance_updated* (poll via *eth_getLogs*, indipendente dallo stato filter del nodo).

13. scripts/AutoVoter.py

Implementa la "automated approve-everything" strategy richiesta da Spec §1.5. Lo script deve *accorgersi* quando nuove proposte compaiono e votare APPROVE.

Bootstrap (ensure contributor): se *auto_voter* non è ancora contributor (deposits=0), fa un deposit da *AUTO_VOTER_DEPOSIT* ETH (default 0.1) — necessario perché *vote()* richiede *isContributor(addr)*.

Loop di polling: ogni *POLL_INTERVAL* secondi (default 5) chiama *events.ProposalSubmitted().get_logs(from_block=last_block, to_block=head)*. Per ogni evento estrae il *proposalId* e invoca *vote_approve()*.

vote_approve: tre pre-check per evitare gas su tx che riverte — *status==Active*, *!hasVotedOn(pid, addr)*, *isContributor(addr)* — poi *vote(pid, true)*. Logging strutturato su stdout e su *data/auto_voter.log*.

Graceful shutdown su SIGINT/SIGTERM. Resilienza a *Web3RPCError* transitori (rinuncia dopo 10 fallimenti consecutivi).

Nota. L'utilizzo di *eth_getLogs* (no filter id) lo rende resiliente ai restart del nodo geth, che invaliderebbero un filter persistente.

14. scripts/GasMeasurement.py

Misura gas per ogni operazione utente del sistema (Spec §1.5). 20 scenari, output su *data/gas_report.csv* + tabella su stdout. Isolamento di stato:

- **BitcoinOracle** riusabile perché lo stato è in mappa con chiave *btcAddressHash* — basta usare un hash diverso per ogni scenario.
- **LendingPool** ri-deployata per ogni gruppo di scenari, altrimenti *_contributorList* e *proposalCount* (entrambi append-only) inquinerebbero la misura di *resolveProposal*, che itera su *_contributorList*.

Scenari coperti (vedi anche §17):

```
deposit (new / existing)
withdraw (partial)
requestOracleUpdate (via pool forward)
submitProposal
vote (approve / reject)
resolveProposal Approved con N = 2, 5, 10 contributori
resolveProposal Rejected (pool low / btc liquidity / weighted vote)
partialRepay (mid / close successful / overpay / on failed)
requestCompensation (first / refilled)
upgradeToAndCall (UUPS v2 swap)
```

Il *requestOracleUpdate* è misurato end-to-end ma *solo lato on-chain*: la fase off-chain dell'oracolo non rientra nel gas-cost che ci interessa qui.

15. Suite Hardhat (test/)

File	Cosa copre
LendingPool.test.js	Init, deposit (acc/min/multi/two), withdraw (CEI + reverts), disposableValue/totalDisposable, onlyActiveLoan mod
Proposal.test.js	submitProposal (id, count, fields, no-contributor-required, rate boundary 1/100, reverts), vote (only contributor, si
Resolve.test.js	Preconditions (non-existent, !applicant, voting period, double resolve, strict >), weighted vote math, early-reject p
Loan.test.js	Costruttore (params, locks, sum), partialRepay waterfall, splitting interest, gain credit diretto, overpay → comp po
Oracle.test.js	update / getBalance / getEthEquivalent, requestUpdate emette UpdateRequested, fee minima, withdrawFees on
OracleFee.test.js	Verifica empiricamente che MIN_ORACLE_FEE == gas(update) × 0.1 gwei (Spec §1.3).
Reentrancy.test.js	1) Attacco su LendingPoolVulnerable: attacker deposita 1 ETH e drena 6 ETH dal pool (1 suoi + 5 rubati) — veri
Termination.test.js	terminate() preconditions (Active revert, Failed-with-owed revert, Successful ok, Failed-fully-settled ok), permissi
Upgradability.test.js	Non-owner non upgrada, owner upgrada con stato preservato bit-a-bit, v2 espone nuova funzione e slot append

16. Flusso end-to-end di un prestito (riepilogo)

La sequenza canonica integra tutti i contratti e ricalca Spec §1.3:

1. contrib_i → LendingPool.deposit(value) (Deposited)
2. applicant → LendingPool.requestOracleUpdate(hash) (UpdateRequested)
oracle_service → BitcoinOracle.update(hash, sat) (BalanceUpdated)
3. applicant → LendingPool.submitProposal(...) (ProposalSubmitted)
4. contrib_i → LendingPool.vote(pid, approve?) (ProposalVoted)
5. wait PROPOSAL_VOTING_PERIOD+1 blocchi
6. applicant → LendingPool.resolveProposal(pid):
 - early-reject (pool/btc) → ProposalRejected
 - weighted yes*2 <= totalDisp → ProposalRejected
 - else: lock proporzionale, sort, new LoanContract{value:loanedAmount} → ProposalApproved + LoanRegistered + LoanCreated
7. applicant ← ETH disbursed (in LoanContract constructor)
- 8a. happy path: applicant → loan.partialRepay(value)*
 - base waterfall → repayLockedValue → unlockValue (in pool)
 - interest → creditInterest (to contributor) + collateral → compPool
 - all repaid → loan Successful → decreaseCollateral, markLoanClosed
- 8b. fail path: applicant non rimborsa, expiry passato → contrib può
loan.requestCompensation():
 - prima volta: status→Failed, increaseCollateral (+5)
 - paga min(owed, compPool) e segna alreadyCompensated
9. (opzionale) chiunque → loan.terminate() (preconditions ok)

17. Risultati di gas

Estratto da *data/gas_report.csv* (run su privatechain Clique, gasPrice 1 gwei):

Operazione	Scenario	Gas	Costo (ETH)
deposit	new contributor	141 091	0.000141
deposit	existing	42 421	0.000042
withdraw	partial	49 984	0.000050
requestOracleUpdate	via pool forward	40 973	0.000041
submitProposal	valid	184 398	0.000184
vote	approve	123 903	0.000124
vote	reject	59 675	0.000060
resolveProposal	Approved (N=2)	1 857 149	0.001857
resolveProposal	Approved (N=5)	2 162 796	0.002163
resolveProposal	Approved (N=10)	2 678 306	0.002678
resolveProposal	Rejected (pool low)	63 139	0.000063
resolveProposal	Rejected (btc)	73 084	0.000073
resolveProposal	Rejected (vote)	82 345	0.000082
partialRepay	mid	78 805	0.000079
partialRepay	close Successful	152 272	0.000152
partialRepay	overpay	155 052	0.000155
partialRepay	on Failed loan	147 364	0.000147
requestCompensation	first call	114 994	0.000115
requestCompensation	subsequent	67 444	0.000067
upgradeToAndCall	UUPS v2 swap	37 736	0.000038

Osservazione: *resolveProposal Approved* ha la crescita più marcata in N perché itera due volte sui contributori (sum + sort) e deploia un nuovo contratto (LoanContract) — la deploy domina il costo fisso. Le tre varianti rejection costano un ordine di grandezza in meno perché escono prima del lock loop.

18. Reentrancy: vulnerabilità + difesa

Spec §1.5 richiede di mostrare come introdurre una vulnerabilità di reentrancy e un test che la sfrutti.

Modifica al codice (*LendingPoolVulnerable.withdraw*):

```
function withdraw(uint256 amount) external {
    require(amount > 0, "Zero amount");
    require(disposableValue(msg.sender) >= amount, "Insufficient disposable");
    (bool ok, ) = msg.sender.call{value: amount}(""); // <-- BEFORE state update
    require(ok, "Transfer failed");
    unchecked {
        deposits[msg.sender] -= amount;
        totalFundingPool -= amount;
    }
}
```

Attacco (vedi *test/Reentrancy.test.js*):

- Alice/Bob depositano 5 ETH ciascuna (pool 10 ETH).
- Attacker deposita 1 ETH (pool 11 ETH, deposits[attacker]=1).
- attacker.attack() → pool.withdraw(1 ETH) → call al receive() → re-entry withdraw(1 ETH) × 5. Ogni nested call vede *deposits[attacker]=1* perché il decremento non è ancora avvenuto.
- Esito: attacker estrae 6 ETH (1 propri + 5 rubati). Pool resta a 5 ETH; Bob non può più ritirare.

Difesa: la versione production di *LendingPool.withdraw* ha (1) modifier *nonReentrant* che imposta *_reentrancyStatus=2* prima del body, (2) CEI corretto (state-update PRIMA della call). Il test verifica che lo stesso attaccante su *LendingPool* sicuro fallisce con "Transfer failed" (la nested withdraw riverte sul guard, il revert si propaga attraverso il low-level call).

19. Strategie malevole non-reentrancy

Spec §1.5 chiede di discutere se esista una strategia maliziosa che permetta a un contributor di trarre vantaggio sleale, sfruttando la majority-rule del rimborso.

Strategia identificata: un attaccante che controlla la *maggioranza pesata* della pool può manipolare l'ordine di saturazione del waterfall a proprio favore.

Esempio: contributor A controlla 80% della disposable, B il 20%. Per ogni proposta, A approva solo quelle in cui la sua *share* calcolata sarà la maggiore (è quasi sempre vero data la dominanza), garantendosi di essere il primo nella lista DESC di *_sortContributors*. Per il rimborso base parziale, A è sempre saturato prima di B. Quindi:

- In caso di prestito Successful, A e B vengono ripagati uguali (entrambi saturati a fine);
- In caso di prestito Failed con rimborso solo parziale (es. 50%), A recupera il 100% della sua quota, B il 0%. Se anche la compensazione compPool non basta, B subisce tutta la perdita.

Il payoff atteso di A è strictly superiore a quello di B, ma A non ottiene *più* capitale di quanto avesse depositato — semplicemente subisce meno volatilità. Non è quindi una *theft*, ma una *risk transfer* sleale fra contributor.

Una difesa possibile sarebbe ribaltare la regola di waterfall in modo *round-robin proporzionale* (ad ogni slice, saturare in proporzione alla quota residua di ciascuno). La Spec, tuttavia, prescrive esplicitamente l'ordine DESC initial-locked (Spec §1.3), quindi questa asimmetria fa parte del modello.

20. Manuale d'uso operativo

Dipendenze necessarie: Node ≥ 20, Python 3.12+, geth, librerie pip (*web3*, *python-bitcoinlib*, *python-dotenv*).

Avvio (3 terminali)

```
# Terminal 1: nodo geth Clique
rm -rf data/geth data/geth.ipc data/accounts.json data/oracle_contract_info.json
```



```
geth --datadir data init project2526genesis.json
geth --datadir data --networkid 202526 \
  --http --http.api eth,net,web3,personal,debug,admin --http.corsdomain "*" \
  --allow-insecure-unlock --nodiscover --maxpeers 0 \
  --mine --miner.gaslimit 30000000 \
  --miner.etherbase 0xd278d247A52C550508ea2b2C9321d816238fb523 \
  --unlock 0xd278d247A52C550508ea2b2C9321d816238fb523 \
  --password 0xd278d247A52C550508ea2b2C9321d816238fb523psw.txt
```

```
# Terminal 2: compile + setup + oracolo
source venv/bin/activate
npx hardhat compile
python3 scripts/InitialSetup.py
OPERATOR_PRIVATE_KEY=$(jq -r .oracle_operator.key data/accounts.json) \
  python3 oracle/oracle_service.py
```

```
# Terminal 3: demo
source venv/bin/activate
python3 scripts/DemoOperations.py
```

Test Hardhat

```
npx hardhat test
```

Misura gas

```
# come sopra ma in terminal 3:
python3 scripts/GasMeasurement.py
# Output: data/gas_report.csv
```

AutoVoter

```
python3 scripts/AutoVoter.py
# (env opzionali: RPC_URL, POLL_INTERVAL, START_BLOCK, AUTO_VOTER_DEPOSIT)
```

Output runtime principali:

- *data/accounts.json* — tutti gli account + chiavi (mai committare).
- *data/oracle_contract_info.json*, *data/lending_pool_info.json* — address+abi consumati dagli script.
- *data/demo_log.txt* — copia testuale dell'output DemoOperations.
- *data/gas_report.csv* — output GasMeasurement.
- *data/auto_voter.log* — log strutturato AutoVoter.

Documento finale. Per il riferimento normativo completo, consultare docs/v011_ProjectP2PBC_LAB2526.pdf.